REPORTE DE PRACTICA DEL TEMA 1

1. PROGRAMACIÓN KOTLIN: DESDE VARIABLES HASTA HERENCIA
2. CONSTRUCCION DE UN LOGIN Y MUESTRA UNA ACTIVIDAD CON WIDGETS
3. CONSTRUCCIÓN DE DIÁLOGOS Y MENÚS
4. PROPUESTA DE UN APP SOLAMENTE CON LOS TEMAS VISTOS EN CLASE

ALUMNO: DEL ROSARIO CASTILLO IRVING

PROFESOR: IVAN AZAMAR PALMA

GRUPO: 4951



INDICE

INTRODUCIÓN 3
MARCO TEORICO 4
1. Android Studio ..... **¡Error! Marcador no definido.**
2. Kotlin ..... **¡Error! Marcador no definido.**
3. Menús ..... **¡Error! Marcador no definido.**
4. Diálogos ..... **¡Error! Marcador no definido.**
5. Widgets más comunes ..... **¡Error! Marcador no definido.**
DESARROLLO 6
1 . PROGRAMACIÓN KOTLIN: DESDE VARIABLES HASTA HERENCIA 6
2. CONSTRUCCION DE UN LOGIN Y MUESTRA UNA ACTIVIDAD CON WIDGETS 8
3. CONSTRUCCIÓN DE DIÁLOGOS Y MENÚS 12
4. PROPUESTA DE UN APP SOLAMENTE CON LOS TEMAS VISTOS EN CLASE 17
2. Menú principal 18
3.Sección de Boletos 19
4. apartado de eventeos 20
Resultados 22
CONCLUSIÒN 25
BIBLIOGRAFIAS 25

INTRODUCIÓN

El **desarrollo de aplicaciones móviles** se ha consolidado como uno de los sectores de mayor crecimiento en el campo de la tecnología de la información. Actualmente, los dispositivos móviles son fundamentales en la vida diaria de millones de personas, quienes dependen de estas aplicaciones para una variedad de tareas, incluyendo comunicación, aprendizaje, trabajo y entretenimiento. Por esta razón, adquirir habilidades para programar aplicaciones móviles utilizando herramientas y lenguajes modernos y eficientes es crucial para los estudiantes de sistemas computacionales.Durante el curso, se han cubierto temas esenciales que establecen las bases para construir aplicaciones funcionales, bien organizadas y con una **interfaz de usuario intuitiva**. **Kotlin**, reconocido como el lenguaje oficial para el desarrollo de Android, ha sido uno de los principales focos. Kotlin se distingue por su **sintaxis limpia**, su total **compatibilidad con Java** y su enfoque de **orientación a objetos**, lo que permite aplicar conceptos como variables, clases y herencia para crear código lógico y reutilizable.

El ciclo de prácticas no solo abarcó la codificación, sino también el diseño de interfaces de usuario. Se realizaron ejercicios clave sobre la implementación de **widgets** (como botones, campos de texto y etiquetas) que son la base de cualquier **interacción fluida** con el sistema. Un hito importante fue la creación de un **módulo de inicio de sesión**, que sirvió como introducción a la gestión de datos de entrada y la autenticación de usuarios. Además, se exploraron recursos como los **menús y diálogos**, componentes que mejoran significativamente la experiencia del usuario al estructurar las opciones de forma jerárquica y al solicitar confirmaciones críticas para acciones importantes.

Estos elementos no solo facilitan el dominio del lenguaje, sino que también contribuyen a la creación de aplicaciones más ordenadas y escalables. Adicionalmente, el curso incluyó prácticas sobre el diseño de interfaces de usuario mediante el uso de **widgets** (como botones, campos de texto y etiquetas) que facilitan la interacción con el sistema. Un ejercicio clave fue la implementación de un **módulo de inicio de sesión**, un componente básico para la autenticación de usuarios en la mayoría de las aplicaciones. También se exploraron recursos como los menús **y diálogos**, que mejoran la experiencia del usuario al organizar las opciones de forma clara y solicitar confirmaciones para acciones importantes. Para finalizar, el conocimiento adquirido culmina con la propuesta de una aplicación simple que integra todos los temas revisados en el aula. Este proyecto final no solo demuestra la aplicación práctica de los conceptos, sino que también estimula la **creatividad** y la **resolución de problemas.**

MARCO TEORICO

1. Android Studio (Entorno de Desarrollo Android Studio)

Android Studio es el **entorno de desarrollo integrado (IDE)** oficial, diseñado específicamente para optimizar la creación de aplicaciones para el sistema operativo Android. Basado en IntelliJ IDEA, incluye características destacadas como un **editor de código avanzado**, un **emulador integrado** para pruebas, **herramientas de depuración** y el **Layout Editor** para el diseño visual de interfaces. Esta herramienta permite a los desarrolladores integrar recursos como bases de datos, imágenes y bibliotecas externas, facilitando la construcción de soluciones completas. Gracias a su compatibilidad con Kotlin y Java, se ha vuelto indispensable en el desarrollo móvil actual.

2. Kotlin (Lenguaje de Programación Kotlin)

Kotlin es un lenguaje de programación moderno, **conciso y seguro**. Fue desarrollado por JetBrains y posteriormente adoptado por Google en 2017 como el lenguaje prioritario para Android. Una de sus mayores fortalezas es la **interoperabilidad con Java**, lo que permite la migración de proyectos y la reutilización de código sin inconvenientes. Su sintaxis simplificada ayuda a **reducir errores comunes** y facilita el aprendizaje para programadores familiarizados con otros lenguajes orientados a objetos. Sus características incluyen el manejo seguro de valores nulos (con el operador Elvis, ?:), soporte para **programación orientada a objetos**, **herencia de clases** y **programación funcional**. En el ámbito móvil, Kotlin garantiza **eficiencia y escalabilidad**, siendo un pilar en la creación de software actual.

3. Menús (Elementos de Navegación)

Los **menús** son componentes vitales en una aplicación, ya que organizan y facilitan el acceso a las diversas funcionalidades para el usuario. Android provee diferentes modalidades de menús:

- **Menú de Opciones:** Ubicado en la barra de acción, contiene las funciones primarias de la aplicación.
- **Menú Contextual:** Se activa con una pulsación larga sobre un elemento, mostrando opciones específicas relacionadas con ese ítem.
- **Menú Desplegable (Popup Menu):** Aparece junto a un ícono o botón, ofreciendo una lista de opciones rápidas.

El uso estratégico de menús mejora significativamente la **usabilidad** al agrupar acciones de manera accesible y clara.

4. Diálogos (Ventanas Emergentes)

Los **diálogos** son pequeñas ventanas que aparecen sobre la interfaz para comunicar información o para **solicitar una confirmación** al usuario antes de proceder con una acción. Los tipos más comunes son:

- **AlertDialog:** Se utiliza para mostrar mensajes importantes, incluyendo botones de acción (como "Aceptar" o "Cancelar").
- **ProgressDialog:** Indica el estado de avance de una tarea que se está ejecutando en segundo plano.
- **Custom Dialog:** Permite una personalización completa de la ventana emergente con diversos widgets.

Los diálogos son cruciales en aplicaciones móviles para **prevenir errores**, mejorar la interacción con el usuario y confirmar procesos delicados, como la eliminación de datos o el cierre de sesión.

5. Widgets más comunes (Componentes de Interfaz)

Los **widgets** son los componentes visuales de la interfaz gráfica que posibilitan la **interacción** entre el usuario y el software. Algunos de los más utilizados incluyen:

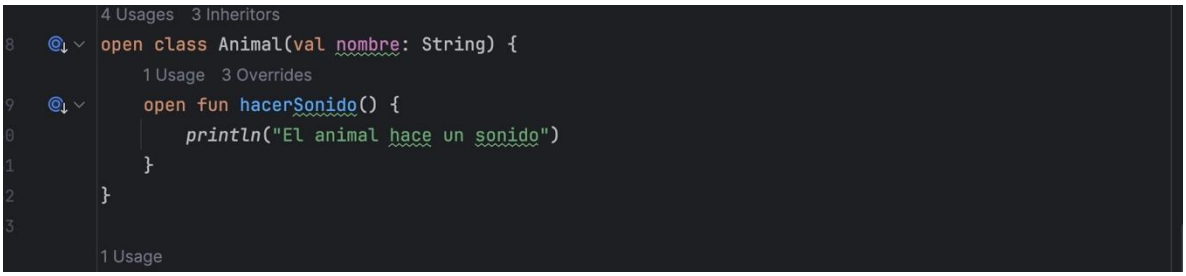
- **TextView:** Se usa para mostrar texto fijo en pantalla.
- **EditText:** Permite al usuario introducir datos, como una contraseña o un nombre de usuario.
- **Button:** Un elemento interactivo que ejecuta una función al ser presionado.
- **ImageView:** Muestra imágenes cargadas desde los recursos de la aplicación.
- **CheckBox:** Permite al usuario seleccionar múltiples opciones de forma simultánea.
- **RadioButton:** Utilizado para seleccionar una única opción de un grupo de alternativas.
- **Spinner:** Presenta un menú desplegable para escoger una opción de una lista.

DESARROLLO

1 . PROGRAMACIÓN KOTLIN: DESDE VARIABLES HASTA HERENCIA

La primera actividad consistió en resolver un ejercicio de programación en Kotlin enfocado en la **herencia** de clases para representar animales.

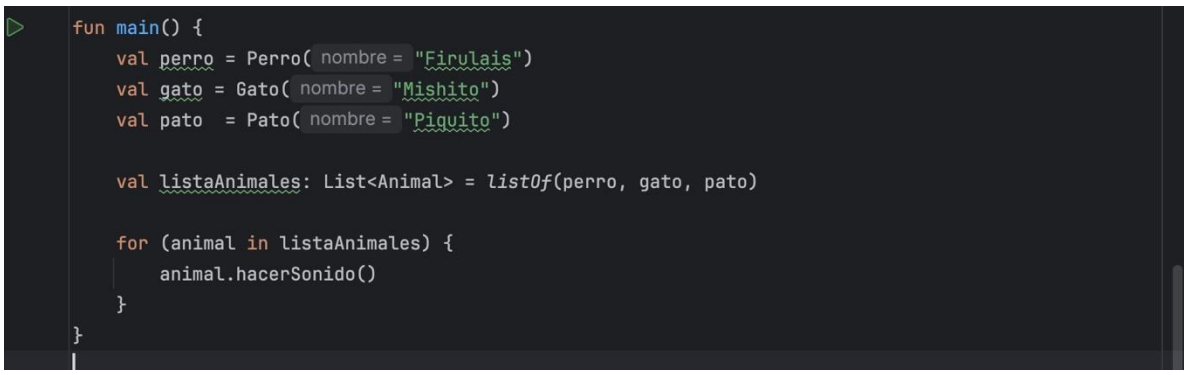
1. Se definió una clase base Animal que contiene la lógica general y de la cual heredarán las clases específicas.



2. Se crearon las **clases hijas** Perro y Gato, que heredan de Animal y personalizan la función hacerSonido().



3. Se declaró la función principal (fun main()) para instanciar variables de cada animal, asignándoles un nombre ("Firulais", "Mishito", etc.).



4. Lo siguiente que vamos hacer es hacer una variable de lista para de los animales que vamos tener que seria perro y gato que antes ya declaramos sus clase y su variable de nombre

```
    val listaAnimales: List<Animal> = listOf(perro, gato, pato)

    for (animal in listaAnimales) {
        animal.hacerSonido()
    }
}
```

5. Despues de todos ya habremos llenado vcada uno de los apartados del codigo para su funcionanmiento de manera correcta
6. lo siguiente seria ahora hacer una nueva clase para otro animal ahora lo cual seria para un pato y que haga lo mismo que las otras dos

```
1 Usage
class Pato(nombre: String) : Animal(nombre) {
    override fun hacerSonido() {
        println("$nombre dice: ¡Cuac!")
    }
}
```

7. Lo siguiente es hacer en la funcion main es agregar al nuevo animal que seria el pato y hacer su variable pato con su nombre

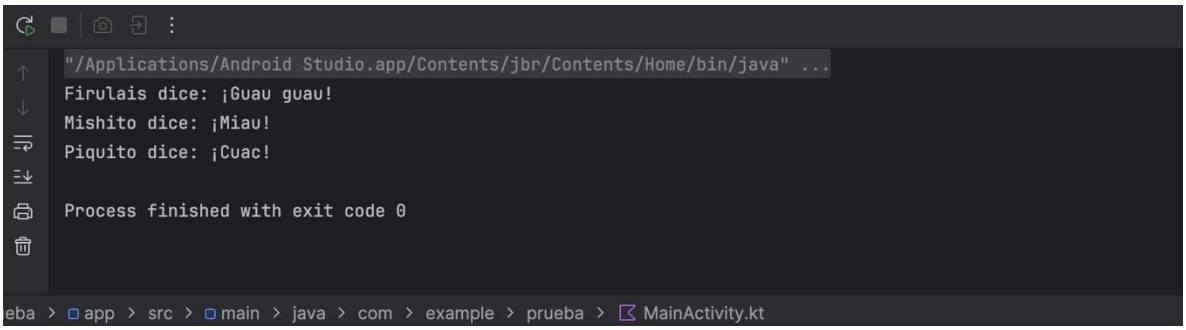
```
    val pato  = Pato( nombre = "Piquito")
```

8. Despues ahora lo que haremos es agrgarlo en la funcion de la lista poner pato para poder aparecer en la lista de los animales que ya tenemos

```
    val listaAnimales: List<Animal> = listOf(perro, gato, pato)

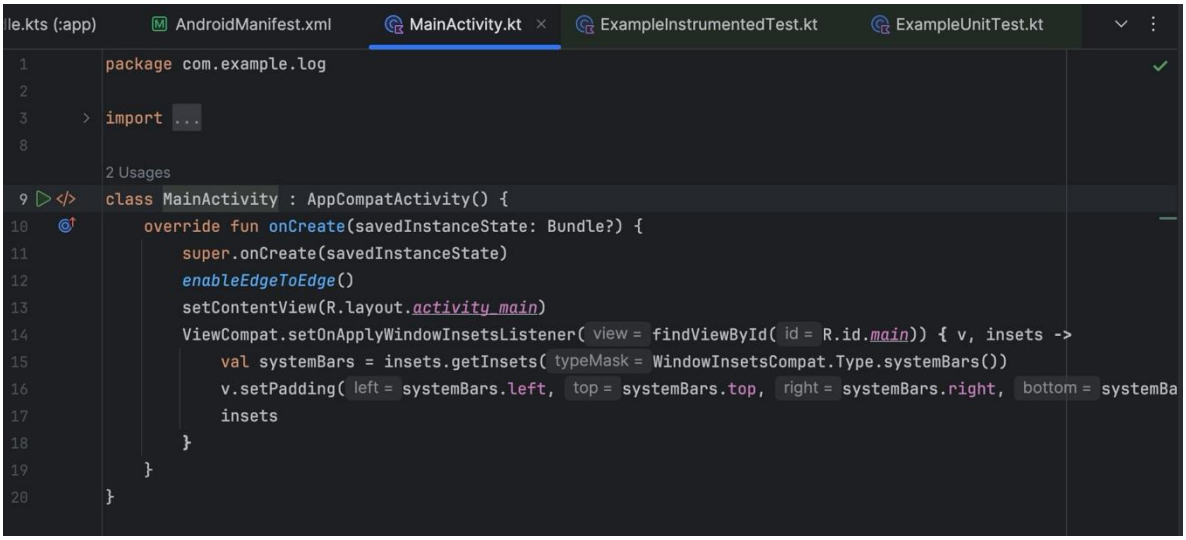
    for (animal in listaAnimales) {
        animal.hacerSonido()
```

9. Ya despues aver hecho todo ahora ya tendremos el codigo con todos los animales que pide hacer su clase lo cual nos mostraria el siguiente resultado



2. CONSTRUCCION DE UN LOGIN Y MUESTRA UNA ACTIVIDAD CON WIDGETS

1. Lo primero que vamos hacer es la creaciòn de nuestra actividad main la cual sera la principal la padre la cual sera que incluye toda la logica de la aplicación



igual lo que tenemos es nuestro manifes de nuestra aplicaciòn ya que describe los componentes principales de la app (actividades, servicios, permisos, etc.) y le dice al sistema cómo debe ejecutarse.

build.gradle.kts (:app) | AndroidManifest.xml | MainActivity.kt | layout/activity_main.xml | ExampleIn

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="LoginApp"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Log">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Text | Merged Manifest

9:43 CRLF UTF-8

2. Lo siguiente que vamos hacer es nuestro exampleinstrumental el cual se crea automaticamente y lo que lleva Este archivo sirve para pruebas instrumentadas, es decir, pruebas que se ejecutan en un dispositivo o emulador Android real, porque necesitan acceso al contexto de la app.

layout/activity_main.xml | ExampleInstrumentedTest.kt | ExampleUnitTest.kt

```kotlin
package com.example.log

import ...

/**
 * Instrumented test, which will execute on an Android device.
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
@RunWith( value = AndroidJUnit4::class)
class ExampleInstrumentedTest {
    @Test
    fun useAppContext() {
        // Context of the app under test.
        val appContext = InstrumentationRegistry.getInstrumentation().targetContext
        assertEquals( expected = "com.example.log",  actual = appContext.packageName)
    }
}
```

3. Lo siguiente es nuestro exampleunittest el cual se quenera solo a la creacciòn de nuestro proyecto android el cual lleva Este archivo sirve para pruebas que corren en la JVM, sin necesidad de un dispositivo o emulador, y normalmente se usa para probar lógica de negocio, funciones o clases de Kotlin.

```kotlin
package com.example.log

import ...

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
class ExampleUnitTest {
    @Test
    fun addition_isCorrect() {
        assertEquals( expected = 4, 2 + 2)
    }
}
```

4. Lo siguiente es nuestro layout activity main . xml el cual es nuestro layout principal este contiene todo nuestro diuseño de la aplicación el cual define nuestro diseño de la interfaz grafica de app

MainActivity.kt | ExampleInstrumentedTest.kt | ExampleUnitTest.kt | activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/bg_gradient"
    android:gravity="center"
    android:orientation="vertical">

    <androidx.cardview.widget.CardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="24dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="8dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="24dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="center"
                android:paddingBottom="16dp"
                android:text="Bienvenido"
                android:textColor="@android:color/black"
                android:textSize="22sp"
                android:textStyle="bold" />

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                app:boxStrokeColor="@android:color/holo_blue_dark"
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    <androidx.cardview.widget.CardView
        <LinearLayout
            <com.google.android.material.textfield.TextInputLayout
                app:boxStrokeColor="@android:color/holo_blue_dark"
                app:startIconDrawable="@android:drawable/ic_dialog_email">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/etEmail"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:hint="Correo electrónico"
                    android:inputType="textEmailAddress" />
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                app:boxStrokeColor="@android:color/holo_blue_dark"
                app:startIconDrawable="@android:drawable/ic_lock_lock">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/etPassword"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:hint="Contraseña"
                    android:inputType="textPassword" />
            </com.google.android.material.textfield.TextInputLayout>

            <Button
                android:id="@+id/btnLogin"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="20dp"
                android:backgroundTint="@android:color/holo_blue_dark"
```

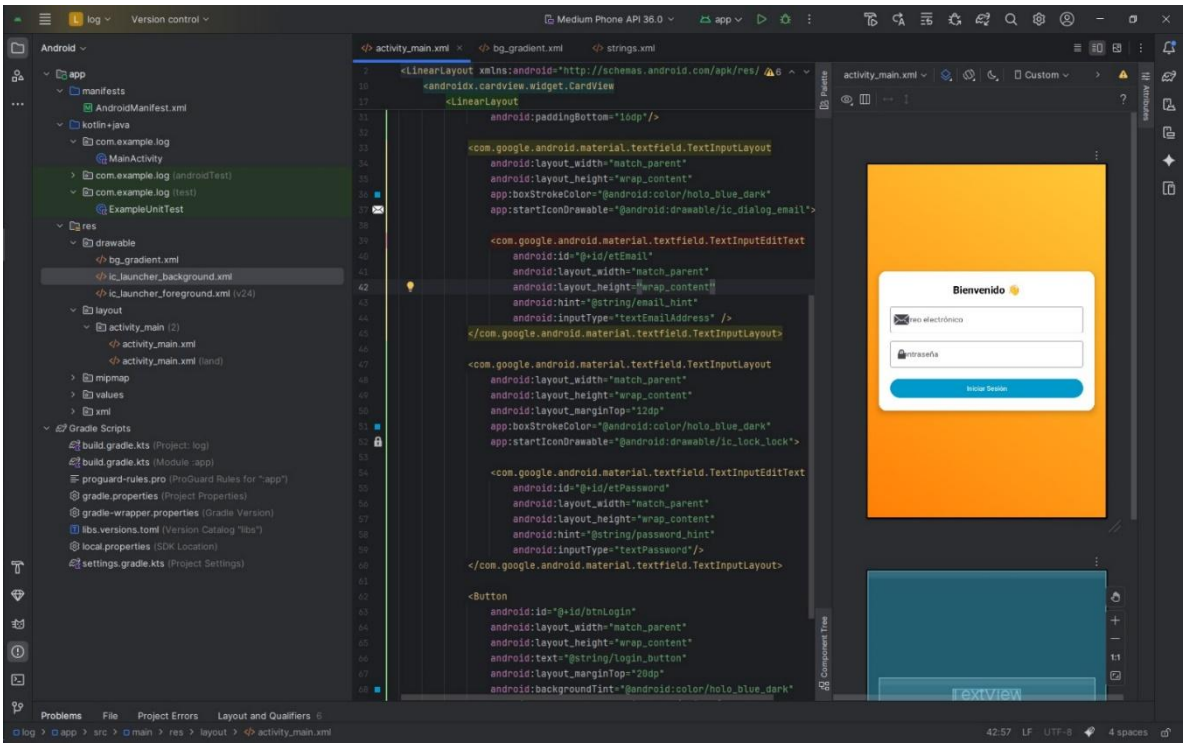
```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    <androidx.cardview.widget.CardView
        <LinearLayout

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                app:boxStrokeColor="@android:color/holo_blue_dark"
                app:startIconDrawable="@android:drawable/ic_lock_lock">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/etPassword"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:hint="Contraseña"
                    android:inputType="textPassword" />
            </com.google.android.material.textfield.TextInputLayout>

            <Button
                android:id="@+id/btnLogin"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="20dp"
                android:backgroundTint="@android:color/holo_blue_dark"
                android:text="Iniciar Sesión"
                android:textColor="@android:color/white" />
        </LinearLayout>
    </androidx.cardview.widget.CardView>
</LinearLayout>
```

5. Por ultimo lo que haremos es probar nuestra aplicación que haya agarrado la creaccion de nuestro login con estilo que le dimos



Aquí como podemos ver nuestro login que agarro de forma corecta y como se ve

3. CONSTRUCCIÓN DE DIÁLOGOS Y MENÚS

Lo primero que hacer es nuestro menu de la apliccaccion que vamos a realizar de nuestras primeras panatllas de la apliucacion El cual la primera parte es el main activity este contendra toda la creaccion del menu de la aplicación y de la primera activity con sus respectivos dialogos

```
package com.ventaboletos.app

import ...

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            VentaBoletosTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    VentaBoletosApp()
                }
            }
        }
    }
}
```

1. Lo siguiente que vamos hacer hay mismo nuestra funcion venta de bolestos la cual es la responsable para la construccion del menu de la parte de venta

```
28

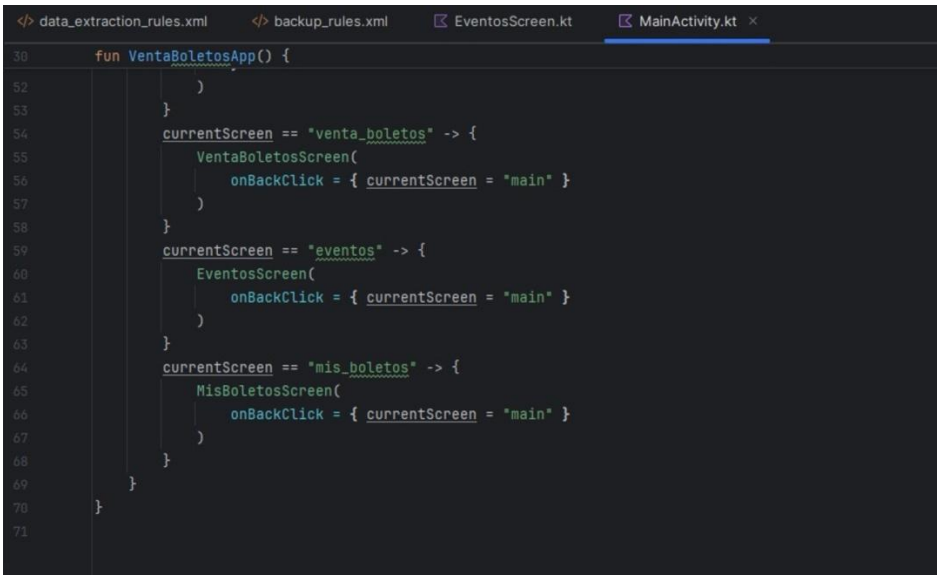
      1 Usage
29    @Composable
30    fun VentaBoletosApp() {
31        var isLoggedIn by remember { mutableStateOf( value = false) }
32        var currentScreen by remember { mutableStateOf( value = "main") }
```

2. Lo que haremos ahora es poner nuestros dialogos en esa ventana de la activity de la venta de bolestos la cual llevar lo siguiente de contedido para especifcar que conllevara

```
34        when {
35            !isLoggedIn -> {
36                LoginScreen(
37                    onLoginClick = {
38                        isLoggedIn = true
39                        currentScreen = "main"
40                    }
41                )
42            }
43            currentScreen == "main" -> {
44                MainScreen(
45                    onNavigateToVentaBoletos = { currentScreen = "venta_boletos" },
46                    onNavigateToEventos = { currentScreen = "eventos" },
47                    onNavigateToMisBoletos = { currentScreen = "mis_boletos" },
48                    onLogout = {
49                        isLoggedIn = false
50                        currentScreen = "main"
51                    }
52                )
53            }
```

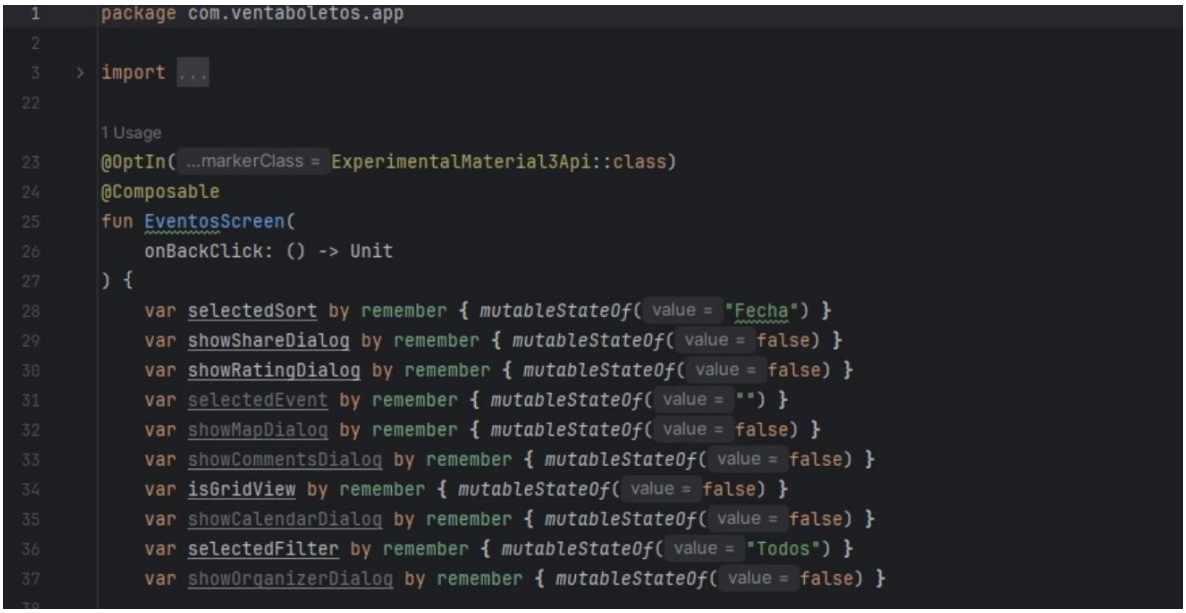
El cual los dialogos son venta de bolestos , eventos y mis boletos que van en la activity de venta

Los siguiente es poner nuestro dialogo en el boton el cual tenemos que validar para que asi nos mande a las siguientes ventanas



3. Lo siguiente que vamos hacer es nuestro menu de la siguiente ventana el cual sera de los eventos disponibles

Lo que haremos es anexar la clase de los eventos con la clase master la cual fue de la primer menu que creamos y hay vamos creando nuestra funcion eventos la cual sera para la realizacion del menu



4. Lo siguinte que haremos es nuestros dialogos que llevara nuestro menu y diferentes apartados de la ventana , los cuales seran los primeros apartados para su descripcion de funcion

```
Row(
    modifier = Modifier.fillMaxWidth(),
    verticalAlignment = Alignment.CenterVertically
) {
    IconButton(onClick = onBackClick) {
        Icon( imageVector = Icons.Default.ArrowBack, contentDescription = "Regresar")
    }
    Text(
        text = "Eventos",
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.weight( weight = 1f)
    )
    IconButton(onClick = { isGridView = !isGridView }) {
```

El siguiente de los principales que tenemos en nuestro menu como los que siguen

```
// Widget 4: Eventos destacados
item {
    Text(
        text = "Eventos Destacados",
        fontSize = 20.sp,
        fontWeight = FontWeight.Bold
    )
}
```

```
154             item {
155                 LazyRow(
156                     horizontalArrangement = Arrangement.spacedBy( space = 12.dp)
157                 ) {
158                     items( count = 3) { index ->
159                         FeaturedEventCard(
160                             title = "Concierto ${index + 1}",
161                             date = "20 Dic 2024",
162                             price = "$${(index + 1) * 100}",
163                             image = "🎵"
```

Nuestro siguiente dialogo es el de filtros rapidos el cual tambien conlleva diferentes funciones de la ventana las cuales filtran los diferentes eventos por alguna categoria

```
// Widget 2: Filtros rápidos
item {
    LazyRow(
        horizontalArrangement = Arrangement.spacedBy( space = 8.dp)
    ) {
        items( items = listOf("Todos", "Hoy", "Esta semana", "Próximo mes", "Gratis")) { filter ->
            FilterChip(
                selected = selectedFilter == filter,
                onClick = { selectedFilter = filter },
                label = { Text( text = filter) }
            )
        }
    }
}

// Widget 3: Selector de ordenamiento
item {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding( all = 16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
```

Nuestro siguiente dialogo que tenemos es de la barra de busqueda

```
// Widget 1: Barra de búsqueda con filtros
item {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy( space = 8.dp)
    ) {
        OutlinedTextField(
            value = "",
            onValueChange = { },
            label = { Text( text = "Buscar eventos...") },
            leadingIcon = { Icon( imageVector = Icons.Default.Search, contentDescription = "Buscar") },
            modifier = Modifier.weight( weight = 1f),
            singleLine = true
        )
        IconButton(
            onClick = { /* Abrir filtros */ },
            modifier = Modifier
                .background(
                    color = MaterialTheme.colorScheme.primary,
                    CircleShape
                )
        ) {
```

5. Como ultimo de nuestros dialogos que tenemos agregados como igual con las diferentes funciones que tiene despues del dialogo como titulo y las funciones con sus respoectivos dialogos de que son

```
// Widget 6: Estadisticas
item {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.primaryContainer)
    ) {
        Column(
            modifier = Modifier.padding( all = 16.dp)
        ) {
            Text(
                text = "Estadisticas del Mes",
                fontSize = 18.sp,
                fontWeight = FontWeight.Bold
            )
            Spacer(modifier = Modifier.height( height = 12.dp))
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceEvenly
            ) {
                StatItem( label = "Eventos"   value = "24"   Icons.Default.Event)
```

4. PROPUESTA DE UN APP SOLAMENTE CON LOS TEMAS VISTOS EN CLASE

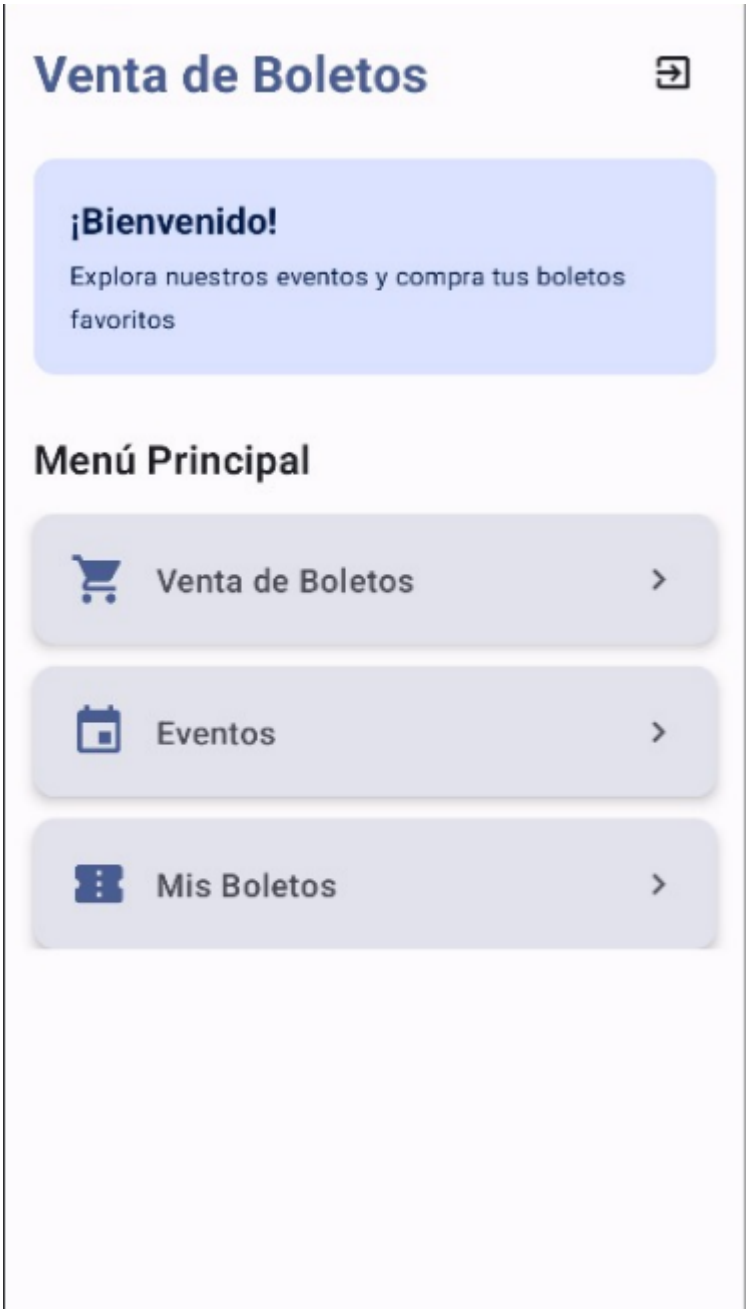
1. Pantalla de Login

La app inicia con una pantalla de inicio de sesión, Aquí el usuario debe ingresar sus credenciales (usuario y contraseña), Una vez que el login sea válido, el sistema redirige a la siguiente actividad, Esta primera parte es básica, pero sirve como filtro para que solo personas registradas puedan entrar. En la imagen que se muestra se puede observar la estructura



2. Menú principal

Menú principal, Después del login, se abre el menú principal, este menú aparece en forma de diálogo, Dentro del menú se encuentran las opciones principales, como: Boletos, Eventos



3.Sección de Boletos

Si el usuario elige “Boletos”, se abre la pantalla para la compra,En esta pantalla se muestran varios elementos:

Lista de eventos: el usuario puede elegir de un desplegable el evento que le interesa.

Categoría del boleto: se ofrecen opciones como VIP, Oro, Plata (por medio de botones tipo radio).

Cantidad de boletos: el usuario escribe o selecciona el número de entradas que desea.



4. apartado de eventeos

Aquí se muestra , wigets y dialogos por ejemplo se muestra el numeri de evento y los datos como la fecha y tambien se muestra el dialogo de texto y se meutras una notificacion donde se hace mencion de un concierrto de rock



También tenemos el apartado de mis boletos donde se muestra los eventos que tenemos, apartado de notificaciones .



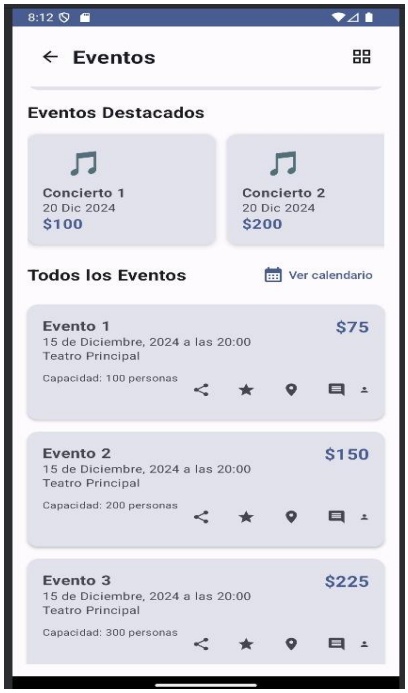
Resultados

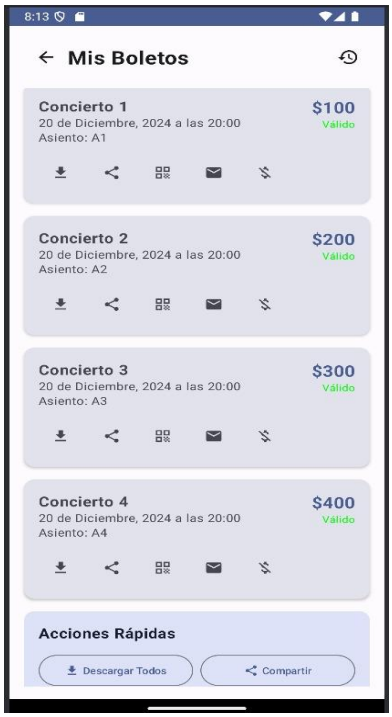
En estas dos imágenes se muestra lo que es el login y el menú principal en los cuales se utilizaron lo diálogos y los widgets





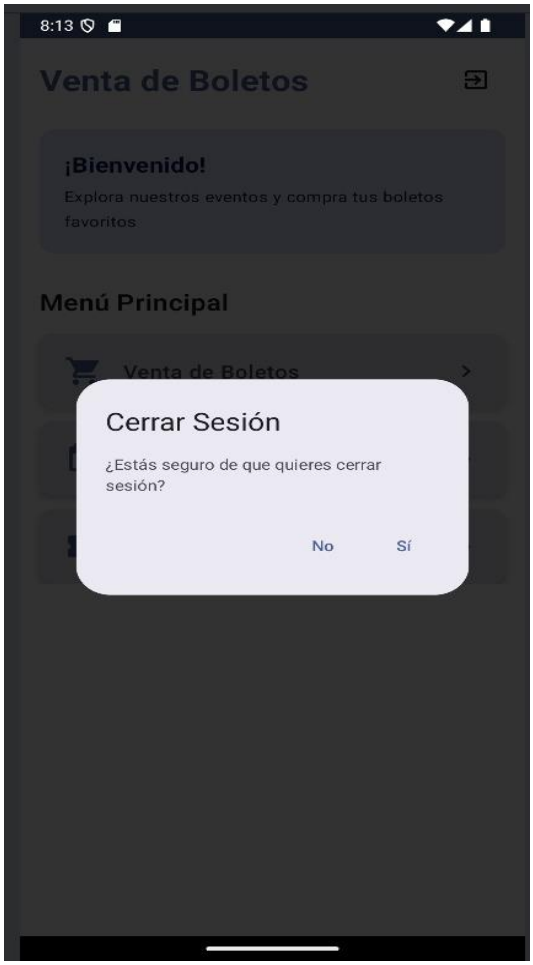
En estas dos imágenes mostramos lo que es el apartado de eventos y mis boletos usando los widgets en la imagen uno se muestra los eventos que se encuentran disponibles y también se muestra el precio de cada boleto y en la imagen dos los boletos que me pertenecen y si aun son vigentes





En estas dos capturas se muestra le apartado de mis boletos pero en este se muestra el evento y la fecha y también se muestra un apartado de recordatorios lo cuales activas para recibir una notificación de ofertas y si el evento esta próximo a llevarse acabo





CONCLUSIÒN

Dominar la programación en **Kotlin** y su aplicación directa en el **desarrollo de aplicaciones móviles** marca un avance fundamental en la trayectoria académica de los estudiantes de sistemas computacionales. Aunque los conceptos explorados en el curso son iniciales, constituyen la infraestructura esencial para crear software funcional y adaptable. La comprensión de temas como el **manejo de variables**, la **programación orientada a objetos** y la **herencia** facilita una codificación ordenada, lo que a su vez optimiza la reutilización y el mantenimiento del código en proyectos futuros. Estos pilares no solo consolidan el conocimiento técnico, sino que también proporcionan herramientas prácticas para afrontar desafíos reales en el ámbito laboral.

La práctica de construir un **módulo de autenticación (login)** fue particularmente relevante, dado que es un componente indispensable en casi todas las aplicaciones modernas. La implementación de **widgets** como los campos de texto y botones ofreció una visión clara de cómo se concreta la **interacción del usuario** con el sistema. Estos componentes visuales, a pesar de su sencillez, son la base para el desarrollo de interfaces más complejas y amigables, impulsando la capacidad del estudiante para balancear la lógica de programación con la **experiencia de usuario (UX)**.

De igual modo, la incorporación de **diálogos y menús** fue de gran valor, ya que subraya la importancia de estructurar la navegación de la aplicación y de validar acciones críticas mediante ventanas de confirmación. Estos elementos mejoran la **usabilidad** y ayudan a prevenir errores comunes, demostrando que la calidad de una aplicación no reside únicamente en el código, sino en la claridad con la que se presentan las opciones al usuario.

Para concluir, la creación de la aplicación propuesta permitió sintetizar y aplicar todos los temas del curso en un proyecto coherente. Este ejercicio estimula la **creatividad**, el **pensamiento crítico** y la **capacidad de resolver problemas**, sirviendo como motivación para una futura especialización en el desarrollo móvil. En resumen, los contenidos revisados son más que una simple introducción a Kotlin y Android Studio; son el punto de partida hacia proyectos más avanzados y la consolidación de habilidades esenciales para el futuro profesional.

BIBLIOGRAFIAS

AndroidDevelopers.(2023). Diálogos.
Google. https://developer.android.com/guide/topics/ui/dialogs

Android Developers. (2023). Widgets comunes de la interfaz de usuario. Google. https://developer.android.com/develop/ui/views/components

JetBrains. (2023). Kotlin programming language. JetBrains. https://kotlinlang.org/

Phillips, B., Stewart, C., Marsicano, K., & Hardy, J. (2019). Android Programming: The Big Nerd Ranch Guide (4th ed.). Big Nerd Ranch.